```
»   config.server_middleware do |chain|
»     chain.add SidekiqMiddleware::Server::SilenceTransientErrors
»   end
    config.death_handlers << ->(job,exception) {
      ErrorCatcherServiceWrapper.new.notify(
        "#{job['class']} won't be retried: #{exception.message}"
```

Since this middleware is somewhat complex, let's write a test for it.

Test Complicated Middleware

To test the middleware, we'll queue jobs as normal, and execute them via `drain_all`, as we did in the system test. That will ensure that the middleware works *and* is being properly used by Sidekiq. Note that Sidekiq does not configure middleware in a test, so the test will need to do that.

We'll want to have three total tests: one that checks that a normal exception bubbles up when the job runs, another that ensures `BaseServiceWrapper::HTTPError` is correctly translated to a `IgnorableExceptionSinceSidekiqWillRetry`, and a third that uses a job class with a custom transient error.

The `setup` block uses code similar to what's in `app/controllers/simulated_behaviors_controller.rb` to provide access to the fake payments service behavior simulation. The second test uses this to cause the fake payment service to crash, thus causing `CompleteOrderJob` to raise `BaseServiceWrapper::HTTPError`.

snapshots/4-4/sidekiq-book/test/lib/sidekiq_middleware/server/silence_transient_errors_test.rb

```
require "test_helper"
require "sidekiq_middleware/server/silence_transient_errors"

class SidekiqMiddleware::Server::SilenceTransientErrorsTest < ActiveSupport::TestCase
  setup do
    @payments_service_status = ServiceStatus.find("payments")
    @payments_service_status.update(sleep: 0, throttle: false, crash: false)
    Sidekiq::Testing.server_middleware do |chain|
      chain.add SidekiqMiddleware::Server::SilenceTransientErrors
    end
```